universidade de aveiro
deti
departamento de eletrónica,
telecomunicações e informática

Distributed Systems
Fault Tolerance

Eurico Pedrosa <efp@ua.pt>

2^{nd} semester - 2025′26

Introduction to Fault Tolerance

Introduction to Fault Tolerance

Dependability is the core requirement for fault-tolerant systems, defined as the degree to which a system can be relied upon to operate as expected

- Distributed systems are characterized by **partial failures**, where one component fails while the rest of the system remains operational
- A fundamental goal of these systems is to **mask such failures** and the subsequent **recovery** from the end user or application
- **Fault tolerance** specifically refers to a system's ability to continue providing its services even in the presence of faults
- Building dependable systems involves four overlapping strategies:
  - preventing, tolerating, removing, and forecasting faults.

Basic Concepts

- Dependability is evaluated through four key properties:
  - **Availability**: The probability that a system is operational and ready for use at any given moment
  - **Reliability**: The property that a system runs continuously without interruption over a specific time interval
  - **Safety**: The assurance that no catastrophic event occurs when a system temporarily fails to operate correctly
  - **Maintainability**: A measure of how easily a failed system can be repaired, often enhanced by automated failure detection

Basic Concepts

- The relationship between system states is defined by the following chain
  - **Fault**: The underlying cause of an error (e.g., a software bug or a loose connector).
  - **Error**: A part of the system state that may lead to a failure (e.g., a damaged data packet)
  - **Failure**: The point at which a system can no longer meet its promises or provide its designated service
- Faults are classified by duration as
  - **transient**: appearing once
  - **intermittent**: recurring unpredictably
  - **permanent**: existing until the component is replaced

Failure Models

Failure types are classified based on the nature of the server's behavior:

Type of failure	Description of server's behavior
Crash failure	The server halts, but works correctly until it halts.
Omission failure	The server fails to respond to incoming requests.
Receive omission	The server fails to receive incoming messages.
Send omission	The server fails to send messages.
Timing failure	The server's response lies outside a specified time interval.
Response failure	The server's response is incorrect (Value or State-transition).
Arbitrary failure	The server may produce arbitrary responses at arbitrary times.

Failure Models: Timing and Response Failures

- **Timing Failures**: Occur when the time of response exceeds the specified interval
  - If a server responds too late, it is specifically called a **performance failure**
  - Essential in real-time systems (e.g., video streaming or control systems)
- **Response Failures**: The content of the response is incorrect
  - **Value failure**: The server provides a response with the wrong data value
  - **State-transition failure**: The server deviates from the correct flow of control upon receiving a request
- **Arbitrary Failures**: Also known as **Byzantine failures**
  - These are the most severe; the server may send contradictory or malicious information to different parts of the system

Failure Models: Failure Detection and System Synchrony

- Detecting failures depends heavily on whether a system is synchronous or asynchronous:
  - **Synchronous systems**: Execution speeds and message-delivery times are bounded
    - Failures can be detected reliably using timeouts.
  - **Asynchronous systems**: No assumptions are made about speeds or delays.
    - It is impossible to distinguish a “crashed” process from an “arbitrarily slow” one
- **Partially synchronous systems**: Behave synchronously most of the time but have periods of asynchronous behavior
  - Timeouts can be used, though they may lead to false positives

Failure Models: Classification of Halting Failures

Failures involving the halting of a process are further categorized by how they are perceived by other processes:

- **Fail-stop**: Crash failures that can be reliably detected (only in synchronous environments)
- **Fail-noisy**: Similar to fail-stop, but the detecting process only **eventually** concludes correctly that a crash happened
- **Fail-silent**: A crash occurs, but external processes cannot distinguish it from an omission failure.
- **Fail-safe**: Arbitrary failures that are benign and cannot cause harm to the system.
- **Fail-arbitrary**: Arbitrary failures that are unobservable and harmful; the most difficult to manage.

Failure Masking

Fault tolerance is achieved primarily through the technique of masking failures, which hides the occurrence of a fault from the user.

- The fundamental principle behind masking failures is the use of **redundancy**
- Redundancy involves providing the system with more resources than are strictly necessary for its basic operation
- By employing redundancy, a system can continue to provide its services even when one or more components fail

Failure Masking: Three Types of Redundancy

- **Information Redundancy**: Adds extra bits to data for the detection and correction of errors
  - e.g., the use of error-correcting codes, such as Hamming codes, during transmission
- **Time Redundancy**: Involves performing an action again if the first attempt fails
  - This is particularly useful for masking transient or intermittent faults
    - e.g., retransmitting a lost network packet
- **Physical Redundancy**: Involves adding extra equipment or processes to the system so that it can function despite component failures
  - e.g., having multiple backup servers or extra hardware circuits

Failure Masking: Physical Redundancy (Hardware vs Software)

- **Physical redundancy** can be applied to both **hardware** and **software** components
- In hardware redundancy, extra circuits or machines are added to take over when a primary unit fails
- In software redundancy (process resilience), extra processes are added to a system to ensure service continuity
- Physical redundancy is the basis for **replication**, a key instrument for improving both efficiency and dependability in distributed systems

Process Resilience

Process Resilience

Process resilience is the primary strategy for masking failures in distributed systems by organizing several identical processes into a **group**.

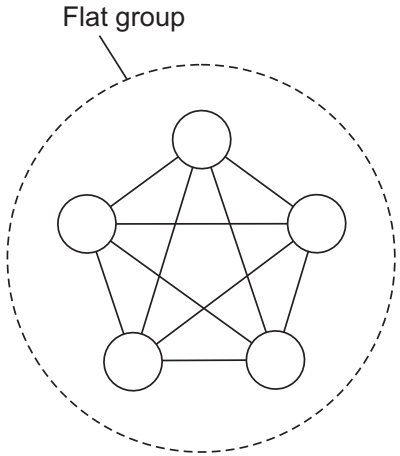
- The fundamental principle relies on software redundancy:
  - if one process in the group fails, another member is available to take over its tasks
- To external clients, the group should ideally be **transparent**, meaning it appears as a single logical process despite being physically distributed across multiple machines

Process Resilience: Process Groups

- The core idea of process resilience is to organize several identical processes into a **group**.
- To the outside world, the group acts as a **single logical entity**
  - a message sent to the group is received by all its members
- If one process in the group fails, another can take over, thereby masking the failure
- **Groups are dynamic**: processes can join or leave the group during the lifetime of the distributed system.

Process Resilience: Group Organization (Flat vs. Hierarchical)

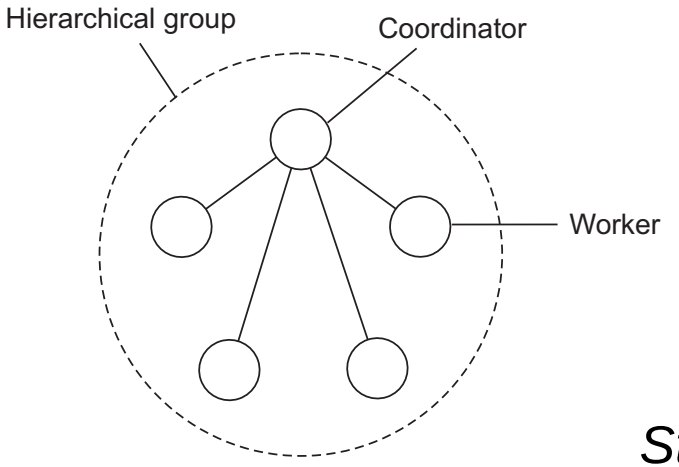
- **Flat Groups**:
  - **All processes are equal**, and decisions are made collectively
  - **Advantage:** There is no single point of failure; if one process crashes, the others remain
  - **Disadvantage:** Complex coordination and overhead increases as the group size grows



Steen and Tanenbaum (2023)

Process Resilience: Group Organization (Flat vs. Hierarchical)

- **Hierarchical Groups**:
  - Organized with a **coordinator** that handles requests and delegates work to workers
  - **Advantage:** Efficient for complex tasks as the coordinator makes primary decisions
  - **Disadvantage:** The coordinator is a single point of failure



Steen and Tanenbaum (2023)

Process Resilience: Group Membership Management

To maintain resilience, a mechanism is required to manage **group membership**. A process must be able to join or leave a group, and the system must detect when a member has crashed.

- Two primary management methods:
  - **Centralized Group Server**: A single server maintains a database of all groups and their current members
  - **Distributed Management**: Membership is managed by the processes themselves using multicasting to notify others of entries and exits
- A major challenge in distributed management is reaching consensus on the membership list (the “view”) when crashes occur during updates

Process Resilience: Group Communication and Addressing

- Redundancy requires that all members of a group receive the same set of messages
- **Addressing Mechanisms**:
  - Processes may use a unique group identifier rather than individual process addresses
  - Hardware multicasting or broadcasting can be used if supported by the underlying network
  - In most distributed systems, the middleware provides an overlay multicasting service to simulate group delivery
- Successful process resilience depends on **Atomic Multicast**, ensuring that either every-one in the group receives a message, or no one does

Failure Masking and Replication

Replication is the key technique used to mask failures by providing process resilience through redundancy.

- The goal is to build a **k-fault tolerant** system, which can survive faults in k components and still meet its specifications
- To achieve this, several identical processes are organized into a group
- When a group is used for masking failures, the number of replicas required depends on the failure model the system must resist
  - **primary-backup** protocol
  - **replicated-write** protocols

Redundancy Requirements in Crash Failures

- If the system only needs to protect against crash failures, it must ensure that at least one process remains operational
- To mask k crash failures, a process group must consist of at least $k + 1$ members
- **Logic:** If k processes crash simultaneously, the remaining 1 process can still provide the service
- This model assumes that processes are **fail-silent**
  - they either work correctly or stop completely

Redundancy Requirements in Arbitrary Failures

Masking arbitrary (Byzantine) failures is significantly more difficult because a faulty process may produce incorrect or conflicting output.

- A system is only resilient if the majority of the group provides the correct result, allowing the system to outvote the faulty members
- To mask k Byzantine failures, a group must consist of at least $2k + 1$ members
- **Logic:** In a group of $2k + 1$ members, even if k members provide “garbage” or malicious results, there are still $k + 1$ members providing the correct result.
- Since $k + 1$ is a majority in a group of $2k + 1$, the correct result can be identified through voting.

Comparison of Replication Bounds

Failure Model	Replica Requirement	Reasoning
Crash Failure	$k+1$	At least one must survive to provide the service.
Byzantine Failure	$2k+1$	A majority is needed to out-vote faulty/malicious nodes.

Assumptions for Masking through Replication

- All replicas must be identical and start in the same initial state
- The system must ensure **Atomic Multicast**: all non-faulty replicas must receive the same set of requests in the same order
- Replicas must be **deterministic**: given the same state and the same input, every non-faulty replica must produce the same output
- If these conditions are met, the state of all non-faulty replicas will remain consistent, allowing the group to mask the failure of any k members

Consensus in Faulty Systems with Crash Failures

Consensus is the fundamental problem where a set of processes must agree on a single data value even if some processes fail or messages are lost.

- In the context of **crash failures**, it is assumed that a process operates correctly until it stops, after which it performs no further actions
- Reaching consensus is critical for coordinating distributed state, such as in leader election or atomic commit protocols.
- The difficulty of the problem varies significantly depending on whether the system is **synchronous** (where bounds on time exist) or **asynchronous**.

Requirements for Consensus

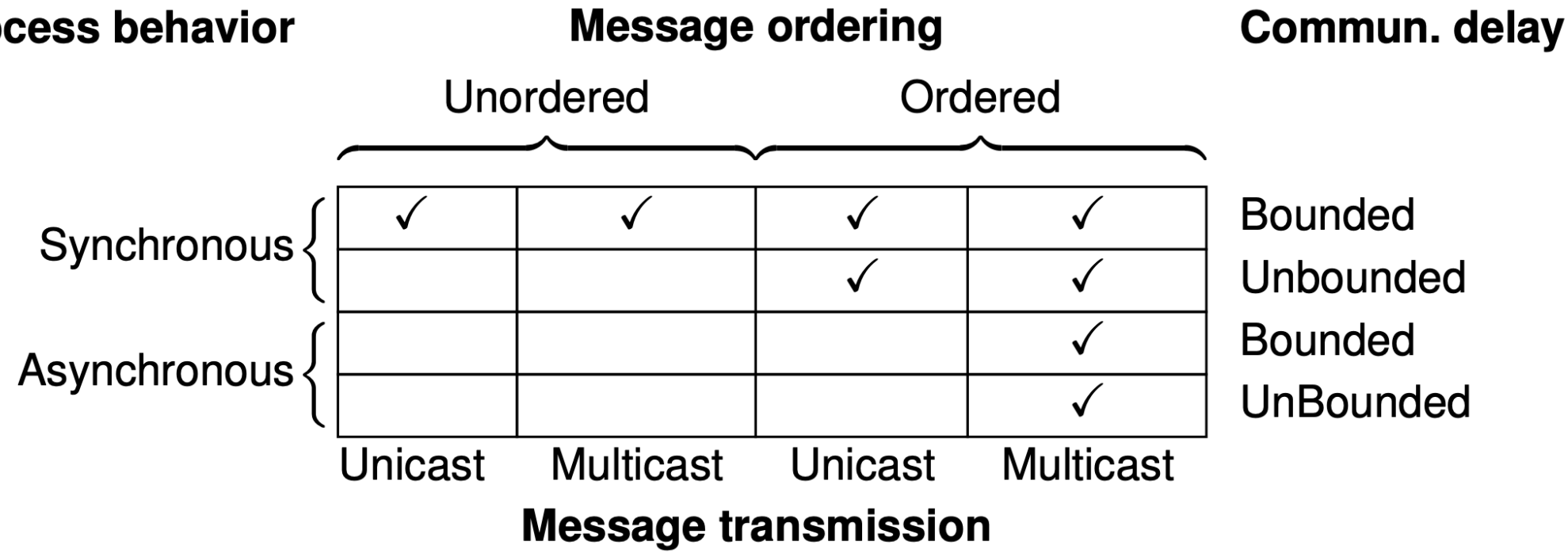
To be considered correct, a consensus algorithm in a system with k faulty processes must satisfy three primary properties:

- **Agreement**: Every non-faulty process must agree on the same value
- **Validity**: If all non-faulty processes proposed the same value v, then the agreed-upon value must be v
- **Termination**: Every non-faulty process must eventually reach a decision and cannot wait indefinitely

The FLP Impossibility Results

A landmark result in distributed computing, established by Fischer, Lynch, and Paterson (FLP 1985), states that consensus is impossible to reach in a fully asynchronous system if even one process crashes.

- Asynchronous systems have no upper bounds on process execution speeds or message delivery times
- The impossibility arises because a process cannot distinguish between a crashed member and one that is simply performing very slowly
- To circumvent this, practical distributed systems rely on timing assumptions (partial synchrony) or randomization



Steen and Tanenbaum (2023)

The CAP Theorem

The CAP theorem states that it is impossible for a distributed system to simultaneously provide all three of the following guarantees:

- **Consistency**: Every read receives the most recent write or an error
- **Availability**: Every request receives a (non-error) response, without the guarantee that it contains the most recent write
- **Partition Tolerance**: The system continues to operate despite an arbitrary number of messages being dropped or delayed by the network between nodes.

The CAP Theorem: Balancing Trade-offs

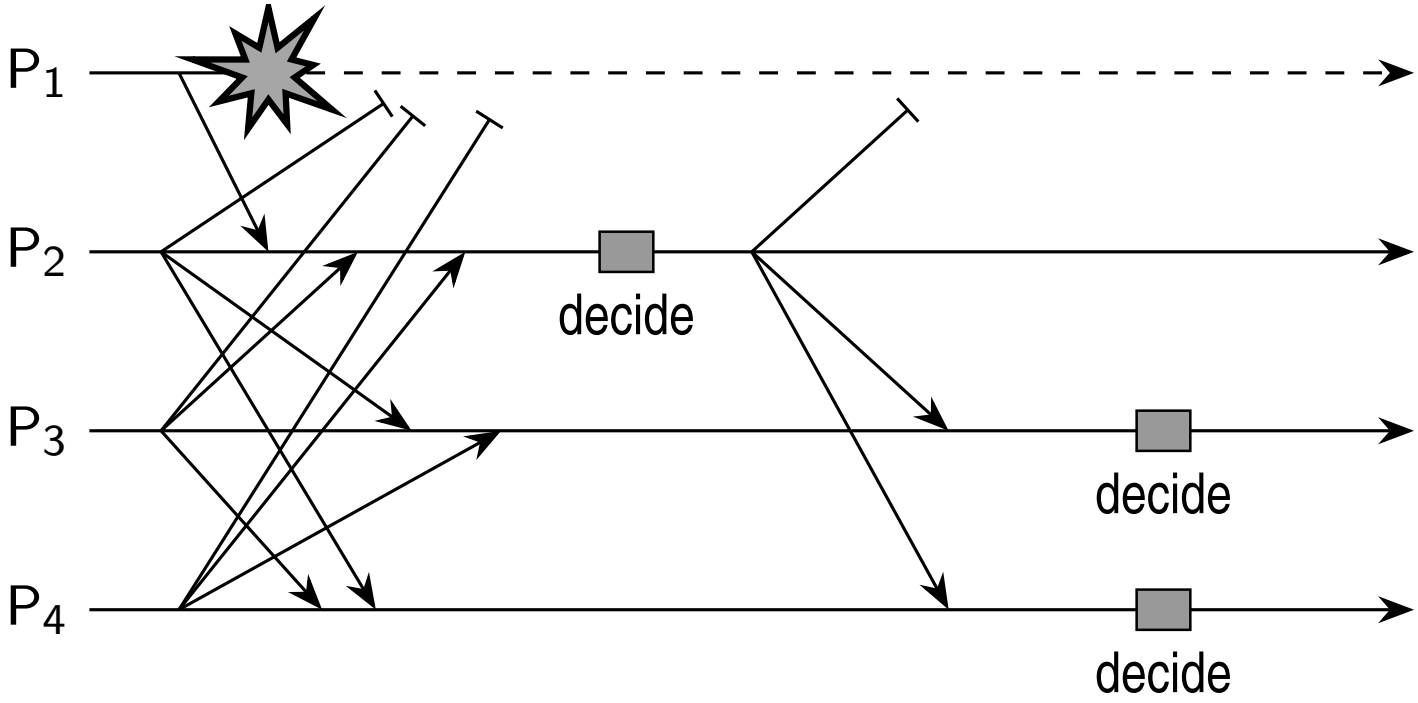
In the presence of a **network partition**, choose between **consistency** and **availability**.

- If **Consistency** is chosen, the system will refuse to respond to some requests to ensure all nodes return the same data, sacrificing availability.
- If **Availability** is chosen, the system will respond to all requests, but different replicas may return different versions of the data, sacrificing consistency
- These trade-offs are central to the **consistency and replication perspective** explored throughout the study of distributed systems

Consensus in Synchronous Systems

- In synchronous systems, processes operate in rounds, and there are known bounds on message delays
- A common algorithm involves processes flooding their proposed values to all other group members: the **flooding consensus**
- If the system tolerates k crash failures, the protocol executes for at least $k + 1$ rounds
- This ensures that at least one round occurs without a new failure, allowing all surviving processes to synchronize their sets of proposed values

Example of Flooding Consensus



Steen and Tanenbaum (2023)

Challenges in Consensus

- The difficulty of consensus is directly proportional to the uncertainty in communication timing and the number of failures to be tolerated
- Crash failures are benign compared to Byzantine failures but still require rigorous algorithmic steps to ensure consistency across replicas
- Most modern fault-tolerant systems are designed to provide consensus in the presence of k crashes using $2k + 1$ total nodes
  - e.g., Paxos or Raft

Introduction to Paxos

- Paxos is a fundamental consensus algorithm introduced by Leslie Lamport, designed to reach agreement in a distributed system with crash failures.
- While it was originally developed in the late 1980s, it remains the basis for many modern fault-tolerant systems
- It is designed to be mathematically rigorous, ensuring that a system can maintain a consistent state even when messages are lost or processes fail.
- Despite its robustness, Paxos is historically noted for its complexity, which motivated the creation of simpler algorithms like Raft

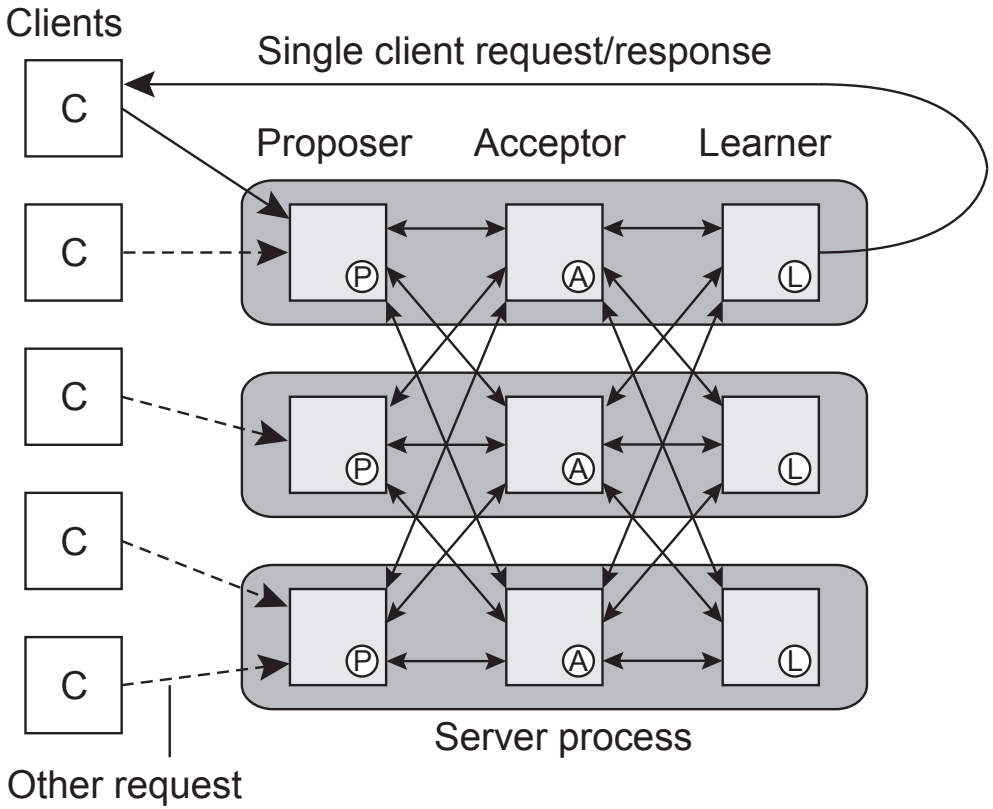
Paxos: Environmental Assumptions

- **Asynchronous Nature**: Paxos can operate in environments with no upper bounds on process speeds or message delivery times
- **Unreliable Communication**: It assumes that messages may be delayed, duplicated, or lost entirely
- **Failure Model**: Processes are assumed to fail by crashing (halting)
  - It does not account for arbitrary (Byzantine) behavior or collusion
- **Persistence**: Processes are expected to have access to stable storage to remember their state across crashes and reboots

Paxos: Logical Roles

In a Paxos implementation, nodes are categorized into three logical roles (which may overlap on a single physical machine):

- **Proposers**: Receive requests from clients and propose them to the group for acceptance
- **Acceptors**: Act as the decision-makers; they vote on proposals and store the chosen values
- **Learners**: Observe the decisions made by acceptors and execute the resulting commands
- **Quorum Logic**: A proposal is only considered “chosen” once it has been accepted by a **majority** of the acceptors



Steen and Tanenbaum (2023)

Paxos: Core Protocol Phases

The Paxos protocol generally proceeds in two distinct phases to ensure safety:

- **Phase 1 (Prepare)**: A proposer selects a unique, monotonically increasing proposal number and asks acceptors to promise not to accept any lower-numbered proposals
- **Phase 2 (Accept)**: If the proposer receives promises from a majority, it sends the actual value to those acceptors
  - If the acceptors haven't made a newer promise, they accept the value and notify the learners
- This two-phase approach prevents different proposers from independently choosing conflicting values

Introduction to Raft

- Raft is a consensus algorithm developed as a more understandable alternative to the Paxos family
- The primary purpose of Raft is to manage a replicated log, which allows a collection of machines to work as a coherent group
- It ensures that the system can continue to make progress as long as a majority of servers are operational and can communicate with each other
- Raft decomposes the consensus problem into three relatively independent subproblems: Leader Election, Log Replication, and Safety.

Raft: Server States

- In a Raft cluster, every server is in one of three mutually exclusive states:
  - **Leader**: Only one leader exists at a time
    - It handles all client requests and manages log replication
  - **Followers**: Only respond to requests from leaders or candidates
    - These nodes are passive
  - **Candidate**: A temporary state used during elections to transition a follower into a leader
- All nodes start as followers and transition to candidates if they do not hear from a leader within a specific timeout period

Raft: Terms and Logical Time

- Raft divides time into **terms** of arbitrary length, which are numbered with monotonically increasing integers
- Terms act as a logical clock in Raft, allowing servers to detect stale information, such as an outdated leader
- Each term begins with an election; if a candidate wins, it serves as the leader for the remainder of that term
- In cases where an election results in a “split vote,” the term ends without a leader, and a new term begins immediately
- Information about the current term is exchanged in every communication between servers

Raft: Leader Election Mechanism

- Raft uses a heartbeat mechanism to trigger leader elections
- Leaders send periodic heartbeats to all followers to maintain authority
- If a follower receives no communication over a period called the **election timeout**, it assumes no viable leader exists
- The follower then increments its current term, transitions to the candidate state, and issues *RequestVote* to all other nodes
- A candidate becomes a leader if it receives votes from a majority of the servers in the cluster

Raft: Log Replication and Commitment

- Once elected, the leader receives commands from clients and appends them to its own log as new entries
- The leader then issues *AppendEntries* in parallel to all followers to replicate the entry
- An entry is considered **committed** once it has been safely replicated on a majority of the servers
- Committed entries are then applied to the state machines of the servers to ensure consistency across the distributed system
- The leader maintains responsibility for consistency
  - It forces follower logs to duplicate its own by overwriting conflicting entries

Consensus in Faulty Systems with Arbitrary Failures

Unlike crash failures, where a process simply stops, processes with arbitrary failures may continue to operate but send contradictory, incorrect, or malicious information to different parts of the system.

- Reaching consensus in this environment is known as the **Byzantine Generals Problem**
  - **The Conflict:** Traitorous generals send conflicting messages to prevent honest generals from reaching consensus.
  - **Agreement:** All honest nodes must decide on the same value.
  - **Validity:** If all honest nodes propose v, they must decide v.
- The primary challenge is to reach agreement among non-faulty processes even when a subset of processes is actively trying to prevent agreement

Consensus in Faulty Systems with Arbitrary Failures

- Research by Lamport, Shostak, and Pease (1982) established the theoretical limits for reaching Byzantine agreement.
- **The Limit**: In a system with n total processes, consensus is possible only if more than two-thirds of the processes are non-faulty.
- Formally, if there are k faulty processes, the system must have at least $n = 3k + 1$ processes to achieve consensus.
- **Example of Failure**: In a 3-node system ($n = 3$) with one faulty node ($k = 1$), consensus is impossible because the two non-faulty nodes cannot distinguish which node is lying when contradictory values are received.

Practical Byzantine Fault Tolerance (PBFT)

The first “efficient” version for high-performance networks; uses a primary/backup system with three phases: **Pre-prepare**, **Prepare** and **Commit**.

The $3k + 1$ Requirement

To tolerate m faulty nodes, the total nodes n must be:

$$n \geq 3k + 1$$

This ensures that any two quorums of size $2k + 1$ overlap by at least $k + 1$ nodes, guaranteeing at least one honest node in the intersection to maintain consistency.

PBFT: The three Phases

1. **Pre-Prepare:** The Leader assigns a sequence number and broadcasts the request
2. **Prepare:** Replicas verify the request and multicast their intent to agree
   - A node enters the “Prepared” state after receiving $2k$ matching messages.
3. **Commit:** Replicas multicast a commit signal
   - Once $2k + 1$ matching commits are received, the request is executed.

If a Leader is detected to be faulty (via timeouts), the replicas execute a **View Change** to elect a new Leader deterministically, ensuring the system maintains **Liveness**.

Failure Detection

Failure detection is a cornerstone of **fault tolerance** in distributed systems. The primary goal is to allow nonfaulty members to decide who is still a member and who has failed.

- Proper failure masking generally requires explicit detection mechanisms
- There are two fundamental mechanisms for detecting process failures:
  - **Active Probing**: Processes send “are you alive?” messages and expect a response
  - **Passive Monitoring**: Processes wait for incoming messages from others
    - only effective if communication is frequent enough
- In practice, most mechanisms boil down to a **timeout** mechanism
  - A process P **suspects** process Q has crashed if Q does not respond within t time.

Failure Detection: Synchronous vs. Partially Synchronous Systems

- **Synchronous Systems**: A suspected crash is equivalent to a known crash
- **Partially Synchronous Systems**: Systems assume **eventually perfect failure detectors**
- Strategy for refinement:
  - If process P suspects Q after time t, but Q later sends a message received by P:
    1. P stops suspecting Q.
    2. P increases the timeout value t to reduce future false positives.

Failure Detection: False Positives and Network Issues

- **Unreliable Networks**: Can lead to false positives where a healthy process is incorrectly removed from membership
- **Crude Timeouts**: Many industry systems rely on single-message timeouts, which are often insufficient for robust detection
- **Gossip-based Detection**:
  - Nodes regularly announce their status to neighbors
  - Availability information is disseminated through the network
  - Processes with “old” availability data are presumed to have failed

Failure Detection: advanced failure subsystems

- **Distinguishing Failures**: A robust system should distinguish between **network failures** and **node failures**
- **Collective Decision**: A single node should not decide a failure alone
  - Upon a timeout, a node may ask other neighbors if they can reach the suspected node
  - Positive information (evidence of life) can be forwarded to cancel false suspicions

Failure Detection: The FUSE Approach

- FUSE utilizes a **spanning tree** for monitoring members in wide-area networks
- Mechanism:
  - Members send ping messages to neighbors in the tree.
  - If a neighbor fails to respond, the detecting node stops responding to its own pings
  - This recursion rapidly promotes a single node failure into a **group failure notification**

Distributed Commit

Ensuring Atomicity in Distributed Systems

- The **distributed commit** problem is a general version of the **atomic multicasting** problem
- It involves ensuring that an operation is performed by every member of a process group, or by none at all
- Examples include message delivery in reliable multicasting or committing a transaction at a single site within a distributed transaction
- These protocols are essential for maintaining data integrity and consistency across multiple networked machines

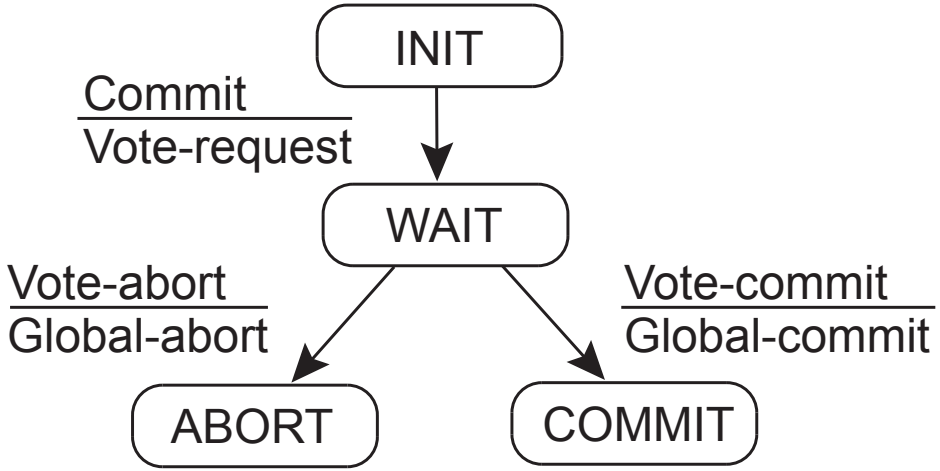
One-Phase Commit (1PC)

- A simple scheme where a central coordinator instructs all participants to perform a specific operation
- **Drawback**: If a participant is unable to perform the operation there is no mechanism to inform the coordinator
  - e.g., due to local concurrency constraints
- Because of this lack of feedback, 1PC is rarely sufficient for complex distributed transactions

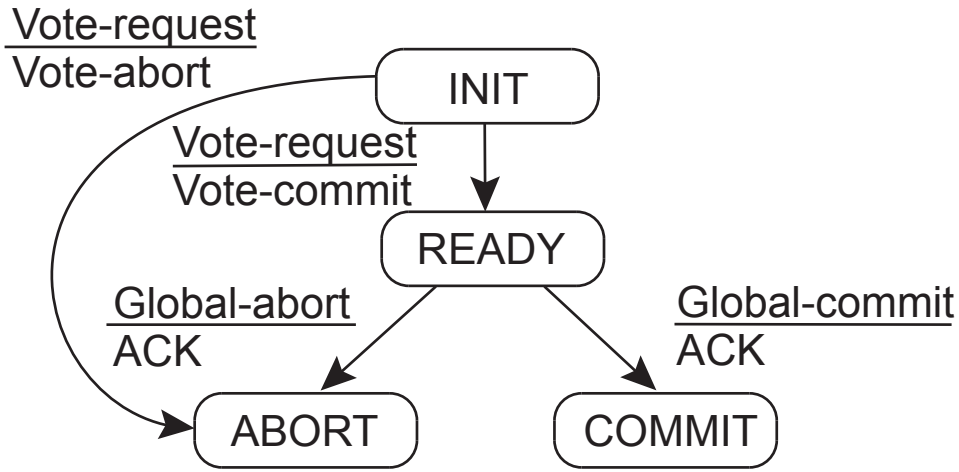
Two-Phase Commit (2PC)

- Developed by Gray (1978) to handle failures more effectively than 1PC
- The protocol is divided into two distinct phases:
  - **Phase 1: Voting Phase**
    - 1. The coordinator sends a `VOTE-REQUEST` to all participants.
    - 2. Participants respond with either `VOTE-COMMIT` (prepared to commit) or `VOTE-ABORT`.
  - **Phase 2: Decision Phase**
    - 3. The coordinator collects votes. If all are `VOTE-COMMIT`, it decides to commit; if any vote is `VOTE-ABORT`, it decides to abort.
    - 4. The coordinator multicasts the global decision (`GLOBAL-COMMIT` or `GLOBAL-ABORT`) to all participants.

Coordinator



Participant



Steen and Tanenbaum (2023)

Two-Phase Commit (2PC): Failure Handling and Timeouts

- Processes may block indefinitely if a participant or coordinator crashes
- **Participant Timeout (INIT state)**: If a participant doesn't receive a `VOTE-REQUEST`, it can safely decide to locally abort and send `VOTE-ABORT`
- **Coordinator Timeout (WAIT state)**: If the coordinator doesn't receive all votes, it must decide to abort and send `GLOBAL-ABORT`
- **Participant Timeout (READY state)**: **Critial**, if a participant in `READY` times out waiting for the coordinator's decision, it must contact other participants to determine the outcome
- **Blocking Nature**: If all operational participants are in the `READY` state, they cannot reach a final decision and must block until the coordinator recovers

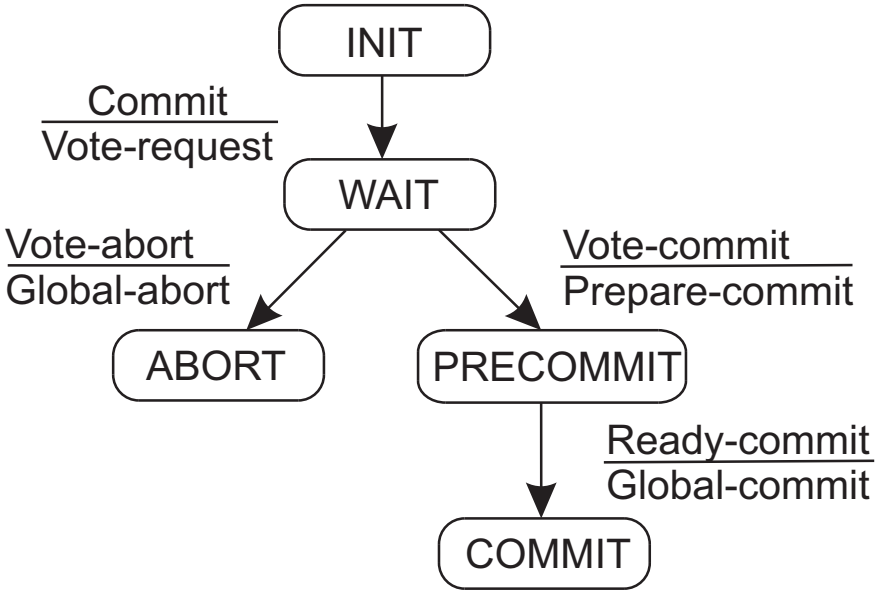
Two-Phase Commit (2PC): Recovery

- Processes must save their state to persistent storage to ensure recovery after a crash
- **Coordinator Recovery**:
  - If crashed in `WAIT`, it retransmits `VOTE-REQUEST`
  - If a decision was already reached and logged, it retransmits that decision
- **Participant Recovery**:
  - If crashed in `INIT`, it can safely abort
  - If crashed in `COMMIT` or `ABORT`, it recovers to that state and retransmits its status
  - If crashed in `READY`, it must contact other processes or the coordinator to learn the final decision

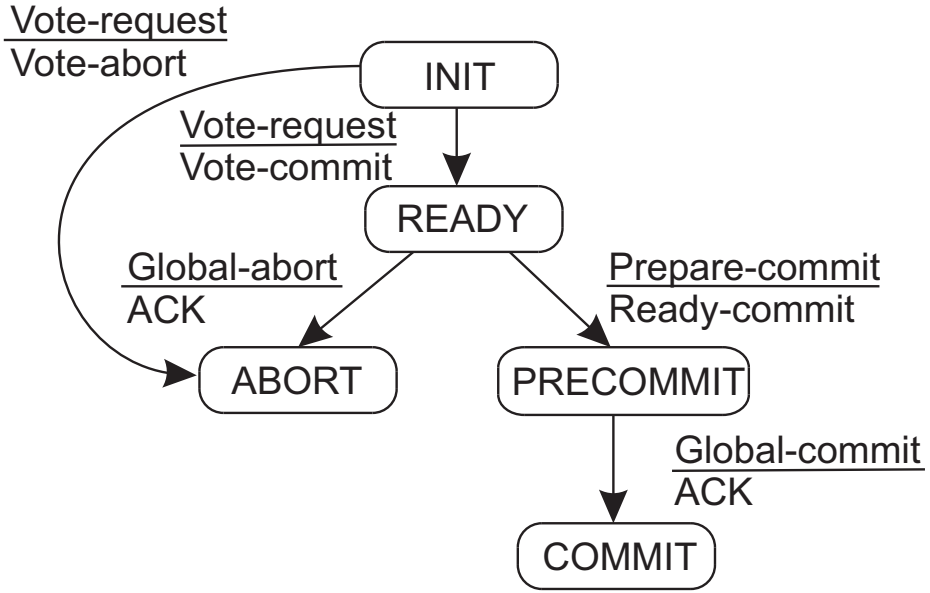
Three-Phase Commit (3PC)

- A non-blocking variant of 2PC designed to avoid process blocking during fail-stop crashes
- It introduces an intermediate `PRECOMMIT` state to ensure the system satisfies two conditions for non-blocking:
  1. No single state can transition directly to both `COMMIT` and `ABORT`
  2. There is no state from which a decision is impossible but a transition to `COMMIT` can be made
- **Mechanism**:
  - If all participants vote to commit, the coordinator sends a `PREPARE-COMMIT` message
  - Only after receiving acknowledgments for the `PREPARE-COMMIT` does the coordinator send the final `GLOBAL-COMMIT`

Coordinator



Participant



Steen and Tanenbaum (2023)

Three-Phase Commit (2PC): Handling Failures

- **Coordinator in PRECOMMIT**: If the coordinator times out, it assumes participants are prepared and instructs operational ones to commit.
- **Participant in READY**: If a participant times out, it contacts others. If any are in `COMMIT`, it commits. If any are in `INIT`, it aborts.
- **Decision Rule**: If a majority of operational participants are in `READY`, the transaction is aborted; if a majority are in `PRECOMMIT`, it is safe to commit.
- Although 3PC avoids blocking, it is less common in practice than 2PC because the specific blocking conditions of 2PC occur infrequently.

Recovery

Introduction

Fundamental to fault tolerance is the recovery from an **error**, which is the part of a system that may lead to a failure.

- The goal of error recovery is to replace an erroneous state with an error-free state.

Two Forms of Error Recovery

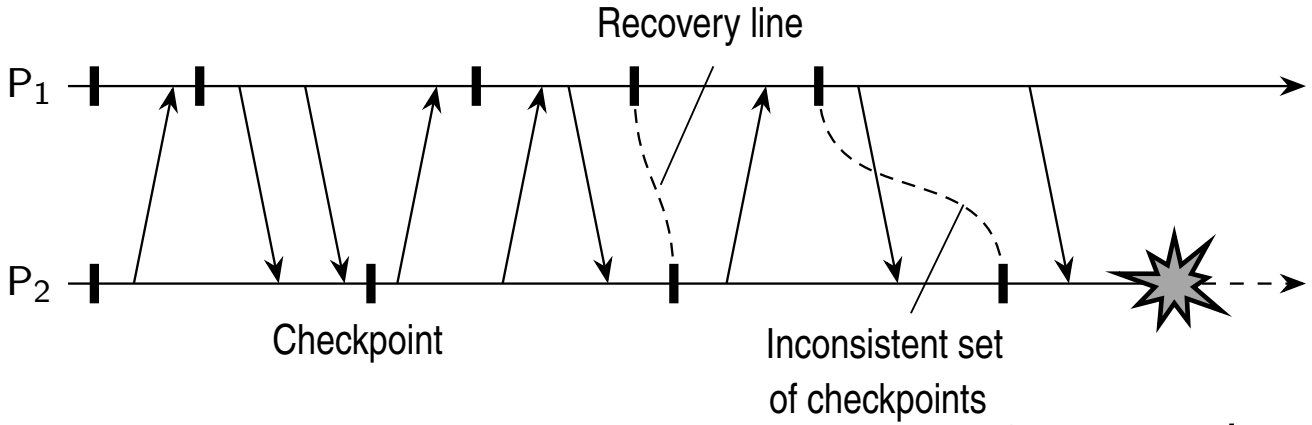
- **Backward recovery**: Brings the system from its present erroneous state back into a previously correct state
  - Requires recording the system's state periodically, known as a **checkpoint**
  - Example: Reliable communication through packet retransmission
- **Forward recovery**: Attempts to bring the system to a correct **new** state from which it can continue
  - Requires knowing in advance which errors may occur to correct them
  - Example: Erasure correction (constructing missing packets from others)

Backward Recovery: Benefits and Challenges

- **Benefits**:
  - Generally applicable method independent of specific systems or processes
  - Can be integrated into the middleware layer as a general-purpose service
- **Problems**:
  - Restoring state is often a costly operation in terms of performance
  - Full failure transparency is hard to provide
    - some applications must be “in the loop”
  - Irreversible actions cannot be rolled back
    - e.g., dispensing cash from an ATM

Checkpointing

- The system must record a **consistent global state** (a distributed snapshot)
- **Consistent Global State**: If process P records receiving a message, process Q must have recorded sending it
- **Recovery Line**: The most recent consistent collection of checkpoints across all processes



Steen and Tanenbaum (2023)

Checkpointing Strategies

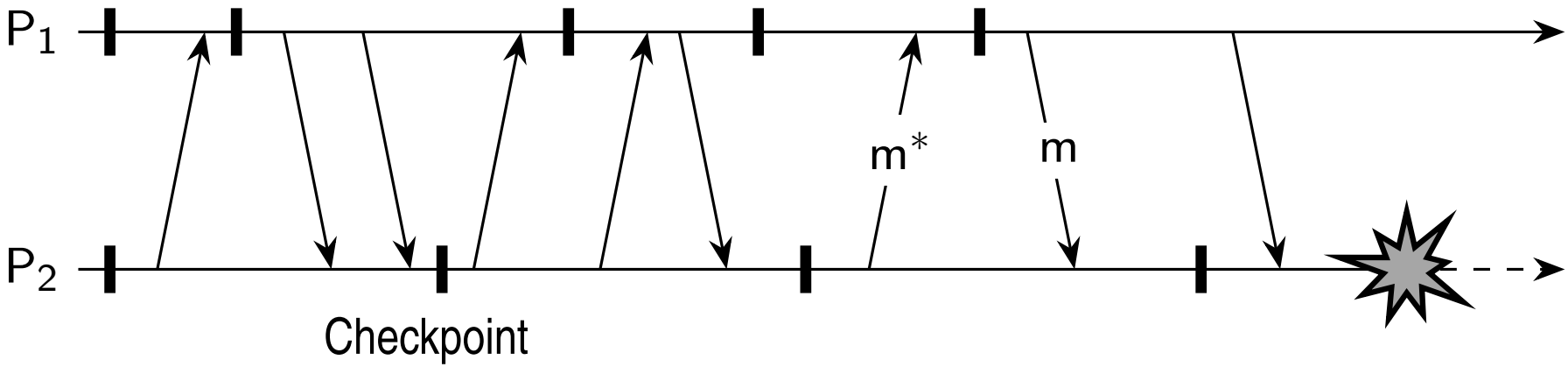
Coordinated Checkpointing

- All processes synchronize to jointly write their state to local storage
- **Mechanism**: A coordinator sends a `CHECKPOINT-REQUEST`
  - processes take local checkpoints, queue outgoing messages, and acknowledge
- **Advantage**: The saved state is automatically globally consistent

Checkpointing Strategies

Independent Checkpointing

- Each process records its local state uncoordinatedly from time to time
- **The Domino Effect**: If local states do not form a distributed snapshot, a cascaded rollback may occur, potentially rolling the system back to its initial state



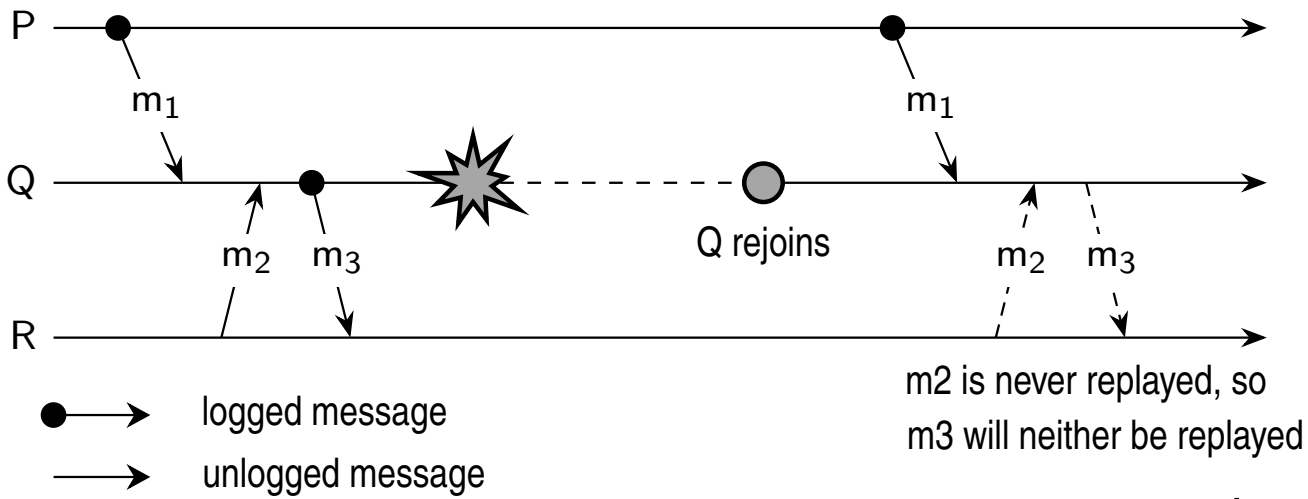
Steen and Tanenbaum (2023)

Message Logging

- Reduces the number of expensive checkpoints by replaying transmitted messages
- **Piecewise Deterministic Model**: Assumes execution is a series of intervals starting with a nondeterministic event followed by deterministic execution
  - e.g., message receipt
- Replaying the same nondeterministic events allows the system to reach the same state.

Message Logging: Orphan Process

- An **orphan process** survives a crash but remains in a state inconsistent with the recovery
- This happens if a process is dependent on a message m that cannot be replayed because its copy was lost in the crash



Steen and Tanenbaum (2023)

Message-Logging Schemes

Scheme	Description
Pessimistic	Ensures each nonstable message is delivered to at most one process. Avoids orphans by blocking until the message is logged.
Optimistic	Allows execution to continue and handles dependencies after a crash by rolling back orphan processes.

- Pessimistic logging is generally preferred in practical distributed systems due to its simplicity compared to optimistic approaches

Resources

Resources

- M. van Steen and A.S. Tanenbaum, Distributed Systems, 4th ed., distributed-systems.net, 2023.